WARWICK

SCHOOL OF ENGINEERING

Lab 2 – Introduction to High Level Synthesis (HLS)

Module Code:

ES4F3/ES3F1

Module Title:

High Performance Embedded Systems Design

Last Update: Jan 2025

Lab 2

Lab 2 is an introduction to HLS (High Level Synthesis) targeting the same Digilent Zybo Z7-20 FPGA board. HLS translates code written in high level languages, C/C++ in this case, to RTL. We'll use HLS to generate a HW component that calculates matrix multiplication. We'll also explore how to take advantage of parallelism in processing using HLS directives (#pragmas) and word lengths affect the FPGA resource utilization by the component and its performance. This documentation covers the process used in this lab.

https://docs.amd.com/r/en-US/ug1399-vitis-hls/Creating-Vitis-HLS-Components

Vitis Setup

Create HLS Component

- Create a new Vitis workspace folder for this lab
- Create a new HLS component File -> New Component -> HLS
- Set the name
- Set configuration file to "Empty File"
- In the source file config click the "Add files" icon next to "DESIGN FILES"
- Add "matrix_mult.cpp" and "matrix_mult.h" from the lab files
- Click "Browse" next to "Top Function" and select "array_mult(stream_data &, data *, stream_data &) matrix_mult.cpp"
- Click the "Add files" icon next to "TEST BENCH FILES" and add "tb_matrix_mult.cpp " from the lab files
- In the next screen, select "Part", search for "xc7z020clg400-1" and select it in the part list
- For the remaining screen keep the defaults and finish
- The sources you added should be in the Vitis components directories

HLS Implementation Flow

- Select the HLS component in the "Flow" section

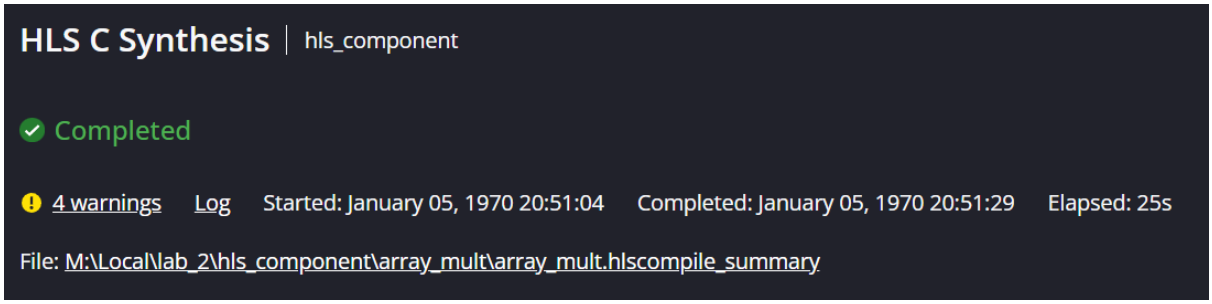
C Simulation

- First run a C Simulation by clicking "Run" under the "C SIMULATION" header in "Flow"
- There may be a popup about the code analyser which should not be used
- This should successfully run the testbench. Two matrices will be generated, an expected multiplication calculated, and an array multiplication calculated.
- A summary is available under the Reports header

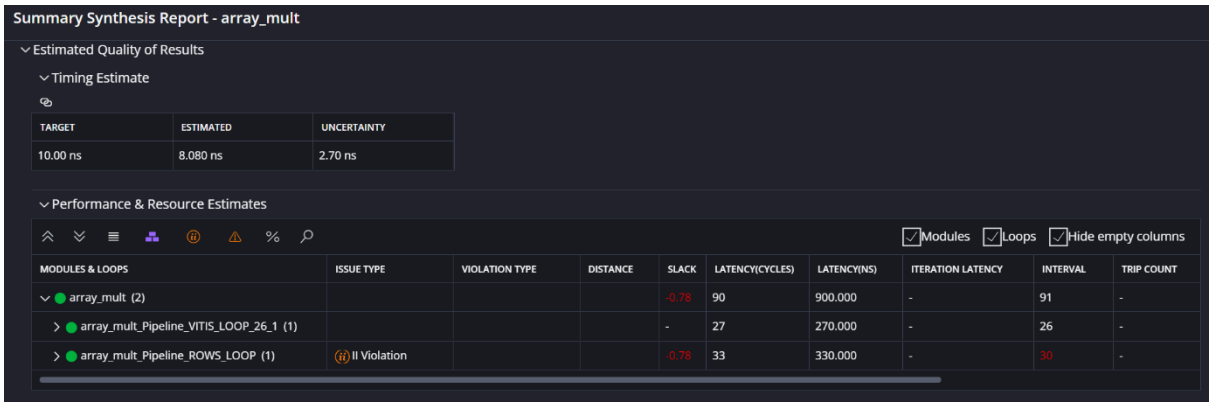
```
Random seed: 1549294172
6  10  3   7   5
10 2   0   0   2
7  0   7   10  2
5  1   6   5   4
0  7   10  2   2
8  9   0   8   8
1  5   8   3   7
0  1   9   0   10
1  4   7   4   9
1  1   2   0   1
Expected array:
70 140 166 106 216
84 102 20  86  96
68 112 137 96  218
50 80  105 63  156
11 55  164 29  169
Array accelerator:
70 140 166 106 216
84 102 20  86  96
68 112 137 96  218
50 80  105 63  156
11 55  164 29  169
INFO [HLS SIM]: The maximum depth reached by any hls::stream() instance in the design is 25
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] *************** CSIM finish ***************
INFO: [HLS 200-112] Total CPU user time: 1 seconds. Total CPU system time: 0 seconds. Total elapsed time: 11.261 seconds; peak allocated memory: 132.273 MB.
INFO: [vitis-run 60-791] Total elapsed time: 0h 0m 15s
C-simulation finished successfully
```

C Synthesis

- Click “Run” under the “C SYNTHESIS” header
- The summary under the results header shows it completed successfully



- There are some warnings in the Kernel Guidance report that can be ignored
- The synthesis report provides:
    - Timing estimate, uncertainty and latency
    - Resource utilisation (BRAM, DSP, FF, LUT, URAM)
    - Information about the AXI interfaces used



TARGET	ESTIMATED	UNCERTAINTY
10.00 ns	8.080 ns	2.70 ns

MODULES & LOOPS	ISSUE TYPE	VIOLATION TYPE	DISTANCE	SLACK	LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT
array_mult (2)				-0.78	90	900.000	-	91	-
array_mult_Pipeline_VITIS_LOOP_26_1 (1)				-	27	270.000	-	26	-
array_mult_Pipeline_ROWS_LOOP (1)	II Violation			-0.78	33	330.000	-	30	-

Co-Simulation

- Click “Run” under “C/RTL COSIMULATION”
- Reports give:
    - Average, max and min latency
    - Total run time

Performance & Resource Estimates

MODULES & LOOPS	AVG II	MAX II	MIN II	AVG LATENCY	MAX LATENCY	MIN LATENCY	TOTAL EXECUTION TIME
array_mult (2)				86	86	86	226
array_mult_Pipeline_VITIS_LOOP_26_1 (1)				25	25	25	
array_mult_Pipeline_ROWS_LOOP (1)				31	31	31	

Package and Implementation

- Click “Run” under “PACKAGE”
- This will package the HLS component as IP which can be used in a Vivado project.
- The IP is stored in a zip folder with the same name as the root function in the HLS component directory of the Vitis workspace.
- Click “Run” under “IMPLEMENTATION”
- This implements the HLS as it would be on the FPGA and reports show full FPGA resource usage

Resource Usage

NAME	VERILOG
SLICE	718
LUT	1348
FF	2190
DSP	15
BRAM	4
URAM	0
LATCH	0
SRL	0
CLB	0

Final Timing

NAME	VERILOG
CP required	10.000
CP achieved post-synthesis	9.670
CP achieved post-implementation	10.155

Tasks

To exploit the parallelism in array multiplication and effectively reduce the latency of our design, we'll use Vivado HLS pragmas to unroll the loop computations. The syntax of the unroll pragma is defined as follows [1]:

```
#pragma HLS unroll factor=2
```

This enables us to partially unroll, by defining a factor, or to fully unroll the loop that the pragma is placed in. If no factor is defined, the pragma unrolls a loop fully. The pragma is placed within the loop we want to unroll.

1) Try different placements of the unroll pragma and with different factors (fully or partial) and synthesize your design. You can also use the pragma in more than one loop.
2) What happens to the synthesis results? (Utilization estimates and latency)
3) You can also explore in the same way the pipeline pragma for various N factors [2]. This pragma effectively reduces the initiation interval between consecutive loop, or function, operations. N should be as minimum as possible.

   ```
   #pragma HLS pipeline II=2
   ```

4) Area savings can also result from reducing the word length of the variables. This can be done by reducing the first parameter in the ap_axis used or declare custom word length variable using ap_int, for signed, and ap_uint for unsigned variables.

More details about HLS interfaces can be found here: https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls/AXI-Adapter-Interface-Protocols

Extension

Resources

[1] - https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls/pragma-HLS-unroll

[2] - https://docs.amd.com/r/2024.1-English/ug1399-vitis-hls/pragma-HLS-pipeline